



Project-IV: Conjugate Gradient Poisson Solver

- Implement the conjugate gradient method (CG) to solve the 2D Poisson equation
 - Measure the performance scaling (i.e., wall-clock time vs. number of cells)
 - Compare the performance with SOR
- Bonus points
 - You get bonus points automatically since topic is not covered by our lecture



這份 **Project-IV: Conjugate Gradient Poisson Solver** 的專題主要目標是實作與比較一種高效的數值解法：**共軛梯度法 (Conjugate Gradient, CG)**，應用於**二維泊松方程 (2D Poisson Equation)**。以下是針對內容的詳細中文說明與任務解析：

專題目標說明 (中文翻譯)

主任務

- 實作共軛梯度法 (CG) 來解 2D 泊松方程：
 - 方程形式通常為：

$$\nabla^2 \phi = f(x, y)$$

在離散化後會變成稀疏線性系統 $A\mathbf{x} = \mathbf{b}$ ，CG 可高效求解對稱正定矩陣 A 。

- 效能評估：
 - 測量在不同解析度 (cell 數量) 下的執行時間，觀察 **scaling behavior (wall-clock time vs. grid size)**。
 - 與傳統的 **SOR (Successive Over-Relaxation)** 方法進行比較，包括：
 - 收斂次數
 - 執行時間
 - 記憶體使用等指標

額外加分項 (Bonus Points)

- 本題並未在課堂中教學，因此只要完成實作就自動加分
- 進一步探索 (如需挑戰)：
 - 更複雜的邊界條件
 - 非均勻網格或 source term

建議實作細節

數值方法步驟 (CG) 簡述

- 建立離散網格與矩陣 A (例如使用五點 stencil)
- 初始化 x_0 , 計算初始殘差 $r_0 = b - Ax_0$
- 設定 $p_0 = r_0$ ，進入迭代：

$$\alpha_k = \frac{r_k^T r_k}{p_k^T A p_k}, \quad x_{k+1} = x_k + \alpha_k p_k$$

$$r_{k+1} = r_k - \alpha_k A p_k, \quad \beta_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$$

$$p_{k+1} = r_{k+1} + \beta_k p_k$$

- 檢查收斂 $\|r_{k+1}\| < \text{tolerance}$

建議比較內容 (與 SOR 比較)

- 使用相同網格尺寸 (例如 $32^2, 64^2, \dots, 512^2$)
- 固定容差，例如 10^{-6}
- 紀錄：
 - 每次迭代的誤差
 - 收斂所需的迭代次數

- 完整計算時間

可視化建議

- 用 contour plot 顯示解 $\phi(x, y)$
- 畫出不同方法下的誤差收斂曲線 (semilog plot)
- 時間 vs. cell 數量的 log-log plot

是否需要我提供 Python 範例程式碼框架 (例如 CG 與 SOR 解法範本) ?

conjugate gradient method是什麼?

Conjugate Gradient Method (共軛梯度法，簡稱 CG) 是一種專門用來高效解對稱正定線性系統 (symmetric positive-definite linear systems) 的迭代方法，常用於大型稀疏矩陣的數值模擬，例如離散化後的 Poisson 方程 或 彈性力學問題。

問題形式

CG 法要解的問題是：

$$\mathbf{Ax} = \mathbf{b}$$

其中：

- A 是對稱且正定矩陣 (Symmetric & Positive-definite)
- $\mathbf{x}$ 是未知向量
- $\mathbf{b}$ 是已知向量

為什麼用 CG ?

- 適用於大型稀疏矩陣，例如 2D/3D Poisson equation 的離散化矩陣。
- 相較於高斯消去法 ($O(n^3)$)，CG 僅需矩陣與向量的乘法 ($O(n)$ 到 $O(n \log n)$)。
- 不需要明確存整個矩陣 A ，只要能定義 $A\mathbf{p}$ 的運算 (例如 stencil operator)。

基本概念與流程

CG 是基於以下觀察：

- 解 $\mathbf{x}$ 也可以看作是在 最小化以下能量函數：

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{Ax} - \mathbf{b}^T \mathbf{x}$$

- 這是一個二次函數，其極小值即為方程 $\mathbf{Ax} = \mathbf{b}$ 的解。

CG 每一步的做法：

1. 初始化 $\mathbf{x}_0$ ，殘差 $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$ ，方向 $\mathbf{p}_0 = \mathbf{r}_0$
2. 重複直到收斂：

- $$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T A \mathbf{p}_k}$$
- $$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$$
- $$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A \mathbf{p}_k$$
- 若 $\|\mathbf{r}_{k+1}\|$ 足夠小則停止
- $$\beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}$$
- $$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$$

「共軛」是什麼意思？

- CG 與 steepest descent (最速下降法) 最大的不同在於，它選擇的方向彼此是A-共軛 (A-conjugate)，也就是：

$$\mathbf{r}_i^T A \mathbf{r}_j = 0 \quad (i \neq j)$$

- $$\mathbf{p}_i \leftarrow \mathbf{p}_j - \alpha (\mathbf{e}_i + \mathbf{J})$$
- 這樣能保證每一步都朝著不同維度的最陡方向前進，使得收斂更快。

效能與收斂性

- 理論上在 n 維問題中 CG 最多 n 步可收斂（在無誤差的情況下）
- 實際上，若特徵值分佈良好，通常只需少量步驟
- 通常可搭配 **preconditioner** 改善收斂速度（稱為 preconditioned CG）

需要我搭配一張圖示（共軛方向的幾何意義）或提供 Python 範例演算法嗎？

↪ 對稱且正定矩陣

對稱矩陣和正定矩陣是什麼？

這是一個非常好的問題，因為要使用 **Conjugate Gradient Method**（共軛梯度法）解線性系統 $A\mathbf{x} = \mathbf{b}$ ，矩陣 A 必須同時滿足兩個條件：**對稱**（symmetric）和 **正定**（positive definite）。我們來分別解釋：

對稱矩陣 (Symmetric Matrix)

一個矩陣 A 是對稱的，當且僅當：

$$A^T = A$$

也就是說：

$$a_{ij} = a_{ji} \quad \text{對所有 } i, j$$

例子：

$$A = \begin{bmatrix} 2 & 3 \\ 3 & 5 \end{bmatrix} \quad \text{是對稱的}$$

正定矩陣 (Positive Definite Matrix)

一個矩陣 A 是正定的，當且僅當對於所有非零向量 $\mathbf{x}$ ，都有：

$$\mathbf{x}^T A \mathbf{x} > 0$$

幾何意義：

這代表 A 對應的二次型 $\mathbf{x}^T A \mathbf{x}$ 是一個開口向上的凸形函數，例如 paraboloid。

例子：

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \Rightarrow \mathbf{x}^T A \mathbf{x} = 2x_1^2 + 3x_2^2 > 0 \quad \text{只要 } \mathbf{x} \neq 0$$

→ 所以是正定的。

對稱正定矩陣 (Symmetric Positive Definite, SPD)

若一個矩陣同時：

- 是對稱的（ $A^T = A$ ）
- 且是正定的（ $\mathbf{x}^T A \mathbf{x} > 0$ 對所有非零 $\mathbf{x}$ ）

就叫做 **對稱正定矩陣**，簡寫為 SPD 矩陣。

Conjugate Gradient 法 就是專門針對這類矩陣所設計的！

應用：為什麼 Poisson equation 對應的矩陣是 SPD？

對於泊松方程離散化後形成的矩陣（例如 5-point Laplacian stencil），會是：

- 稀疏
- 對稱（因為離散 Laplacian 是對稱 operator）
- 正定（因為能量形式 $\int |\nabla \phi|^2$ 對所有非零解都是正的）

很好！讓我們來實際示範把 2D Poisson 方程離散化，看看會如何產生一個具體的矩陣系統 $A\mathbf{x} = \mathbf{b}$ 。

🎯 目標：解 2D Poisson 方程

$$\nabla^2 \phi(x,y) = f(x,y)$$

在區域 $[0,1] \times [0,1]$ 上，使用 3×3 網格點（不含邊界）。所以共有 9 個內部未知點。

1 網格點與編號

我們只處理內部點。對於 3×3 網格，可將每個 (i,j) 點編號成一維向量：

SCSS

複製編輯

(i,j) 坐標 → x_k 向量位置：

(1,1) → x₀

(1,2) → x₁

(1,3) → x₂

(2,1) → x₃

(2,2) → x₄

(2,3) → x₅

(3,1) → x₆

(3,2) → x₇

(3,3) → x₈

2 五點 stencil 差分公式

對於每個內部點 (i,j) ，差分形式：

$$\frac{\phi_{i+1,j} + \phi_{i-1,j} + \phi_{i,j+1} + \phi_{i,j-1} - 4\phi_{i,j}}{h^2} = f_{i,j}$$

這表示每個未知數 $\phi_{i,j}$ 會跟它上下左右相鄰的 4 個點產生關聯。

3 建立矩陣 A (9×9)

矩陣 A 的每一列代表一個 x_k 對應的 Poisson 方程。

$$A = \frac{1}{h^2} \begin{bmatrix} -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -4 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & -4 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & -4 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & -4 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & -4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 \end{bmatrix}$$

- 主對角線是 $-4/h^2$
- 相鄰點是 $1/h^2$
- 非相鄰點是 0
- 邊界條件 (Dirichlet) 會在 b 中處理

4 結果：線性系統

你會得到這樣一個系統：

$$A \cdot \mathbf{x} = \mathbf{b}$$

- $\mathbf{x}$ ：包含 9 個未知數 $\phi_{i,j}$
- $\mathbf{b}$ ：包含 9 個 $f_{i,j}$ ，加上邊界值帶進來的修正

✅ 總結

這就是為什麼「求解 Poisson 方程」會轉換為「求解矩陣方程 $A\mathbf{x} = \mathbf{b}$ 」。

是否需要我幫你畫出這個 3×3 stencil 與網格對應矩陣 A 的視覺圖解？會更直觀地顯示 stencil 與矩陣的關係。

↪ 適用於**大型稀疏矩陣**，例如 2D/3D Poisson equation 的離散化矩陣。

所以矩陣的階數太少就不會滿足大型稀疏的條件？

非常正確的觀察！你可以這樣理解：

✅ 是否「大型稀疏」的判準？

◆ 小型矩陣（例如 3×3 網格 $\rightarrow 9 \times 9$ 矩陣）：

- 是稀疏的（大部分元素為零）✅
- 但不是大型的 ❌
→ 所以不需要用 Conjugate Gradient 這類專為大規模系統設計的迭代法

小型矩陣更適合用直接解法（如 LU 分解、Cholesky）來求解，這些方法在小系統中更準確、收斂快。

◆ 大型稀疏矩陣（例如 1000×1000 網格 $\rightarrow 10^6 \times 10^6$ ）：

- 元素幾乎都是零（每行只有幾個非零數）✅
- 維度太大無法儲存完整矩陣、也不適合做矩陣分解 ❌
→ 就非常適合用 CG、GMRES 等稀疏迭代法來解 ✅

📌 小結：兩個條件要同時滿足

條件	CG 適用嗎？
小型 + 稠密	❌（用 LU 分解更快）
小型 + 稀疏	❌（LU 還能應付）
大型 + 稠密	❌（記憶體爆掉）
大型 + 稀疏 ✅	✅（CG 的用武之地）

你是否有想測試的網格大小？我可以幫你估算一下矩陣大小與非零元素數量，以及是否值得使用 CG。

所以 Poisson equation 的離散化矩陣一定是稀疏矩陣，但不一定是大型矩陣

完全正確！你可以這樣記住：

✅ 結論一句話：

Poisson equation 離散化後的矩陣：
一定是稀疏矩陣 ✅，但
只有在高解析度（大網格）時才是大型矩陣 ✅/❌

◆ 為什麼一定是「稀疏」的？

因為：

- 離散化 Poisson equation 時，只使用了網格點的「鄰近」資料（例如五點 stencil）
- 所以在 A 的每一列中，最多只有 5 個非零元素（對應上下左右 + 自己）
- 其他全是零

這樣的矩陣稱為**稀疏矩陣**（sparse matrix）

◆ 為什麼「不一定是大型矩陣」？

取決於你網格的解析度：

網格大小 $N \times N$	矩陣大小 $N^2 \times N^2$	是否大型矩陣
3×3	9×9	❌ 太小
10×10	100×100	❌ 由小型

$10^2 \sim 10^3$	$10^4 \sim 10^5$	🔴 不建議
100×100	$10^4 \times 10^4$	✅ 中大型
1000×1000	$10^6 \times 10^6$	✅✅ 超大型

📌 實務意義

- 若是小網格，可以直接用 LU / Cholesky 分解解線性系統
- 若是中大型網格（例如 512×512 或更大），就應使用：
 - 共軛梯度法（CG）
 - 多重網格法（Multigrid）
 - 預條件法（Preconditioned CG）

需要我幫你畫出一張圖，顯示稀疏矩陣的視覺化樣子（例如在哪些位置是非零）嗎？這在理解矩陣結構上很有幫助。

👉 * 解 $\mathbf{x}$ 也可以看作是在 **最小化以下能量函數**：
 $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{b}^T \mathbf{x}$
 $\mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{b}^T \mathbf{x}$ 這是一個二次函數，其極小值即為方程 $\mathbf{A} \mathbf{x} = \mathbf{b}$ 的解。

不懂，請詳細說明

你問得非常好，這個觀念是共軛梯度法（CG）背後的數學核心。

我們來一步步從線性代數的角度推導「為什麼解 $\mathbf{Ax} = \mathbf{b}$ 等價於最小化一個能量函數」。

🎯 問題目標

假設我們要解一個線性方程組：

$$\mathbf{Ax} = \mathbf{b}$$

其中：

- $\mathbf{A}$ 是對稱正定矩陣（Symmetric Positive Definite）
- $\mathbf{x}$ 是我們要解的向量
- $\mathbf{b}$ 是給定的向量

💡 想法：能不能把這個問題轉換成一個「優化問題」？

是的！事實上，可以把這個問題轉換為最小化一個二次函數：

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{Ax} - \mathbf{b}^T \mathbf{x}$$

這就是共軛梯度法的數學基礎。

🔍 為什麼這樣的 $f(\mathbf{x})$ 和方程 $\mathbf{Ax} = \mathbf{b}$ 有關？

我們來對這個函數 $f(\mathbf{x})$ 求梯度（gradient）：

$$\nabla f(\mathbf{x}) = \frac{d}{d\mathbf{x}} \left(\frac{1}{2} \mathbf{x}^T \mathbf{Ax} - \mathbf{b}^T \mathbf{x} \right)$$

這裡的兩項分別來看：

- $\frac{1}{2} \mathbf{x}^T \mathbf{Ax}$ 的梯度是 $\mathbf{Ax}$ （因為 $\mathbf{A}$ 是對稱的）
- $-\mathbf{b}^T \mathbf{x}$ 的梯度是 $-\mathbf{b}$

所以整體梯度就是：

$$\nabla f(\mathbf{x}) = \mathbf{Ax} - \mathbf{b}$$

✅ 結論

當我們讓這個函數取得極小值時：

$$\nabla f(\mathbf{x}) = 0 \quad \Rightarrow \quad A\mathbf{x} - \mathbf{b} = 0 \quad \Rightarrow \quad A\mathbf{x} = \mathbf{b}$$

這就說明了：

解線性方程組 $A\mathbf{x} = \mathbf{b}$ · 等價於最小化能量函數 $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T \mathbf{x}$

🎨 幾何圖像 (非常重要！)

- 這個函數 $f(\mathbf{x})$ 是一個 上凸的二次曲面 (形狀像碗)
- 你可以想像在這個碗表面「往最陡的方向下滑」，最後會到達底部
- 底部的位置 (極小值點) 就是滿足 $A\mathbf{x} = \mathbf{b}$ 的解

🔄 與共軛梯度法的關係？

- 共軛梯度法就是一步步「選擇聰明的方向」，朝這個能量函數的極小值前進
- 每一步選擇的方向是「共軛方向」，比最速下降法 (steepest descent) 更有效率

是否需要我畫一張簡單的二維圖來說明這個二次函數的「碗狀結構」與其極小值？這會幫助你更直觀理解。

↪ 等價於最小化能量函數

為什麼不是等價於最大化能量函數，畢竟只是取梯度為零而已，還需要用二階判斷式，判斷是極大值或是極小值

非常銳利的問題，你說得完全正確！
僅僅使用「 $\nabla f(\mathbf{x}) = 0$ 」這個條件，只能說我們找到了一個駐點 (stationary point)，但不能確定是極小值、極大值、還是鞍點。
因此，我們還需要檢查二階導數 (Hessian)，也就是你提到的「二階判斷式」。

✅ 為什麼 $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T \mathbf{x}$ 是極小值？

來分析這個函數的性質：

1 梯度 (Gradient)

剛才我們已經知道：

$$\nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$$

當 $\nabla f = 0$ 時，剛好是 $A\mathbf{x} = \mathbf{b}$

2 二階導數 (Hessian)

函數 $f(\mathbf{x})$ 的 Hessian 為：

$$\nabla^2 f(\mathbf{x}) = A$$

- 這是因為：
- $\frac{1}{2}\mathbf{x}^T A\mathbf{x}$ 對 $\mathbf{x}$ 的二階導為 A
 - $-\mathbf{b}^T \mathbf{x}$ 對 $\mathbf{x}$ 的二階導為 0

3 判斷極值類型：由 Hessian 矩陣決定

這裡出現關鍵！

- 若 A 是正定矩陣，則 $\nabla^2 f = A \succ 0$ ，代表函數 $f(\mathbf{x})$ 是 嚴格凸函數 (strictly convex)
- 凸函數的唯一駐點必為 全域極小值

結論：因為 A 是對稱正定 (symmetric positive definite)，所以 $f(\mathbf{x})$ 在唯一的駐點處取得極小值

❖ 為什麼不是極大值？

如果 A 是 負定矩陣 (即所有特徵值都小於 0)，那麼：

- 函數變成「碗口向下」的二次型
- 此時駐點會是極大值

但我們的前提是：CG 只能處理對稱正定的 A ，這也保證了我們的 $f(\mathbf{x})$ 是上凸、極小值存在且唯一。

← 總結

你說得非常對，極值點 $\neq$ 一定是極小值，除非你：

1. 確認梯度為 0 (駐點)
2. 確認 Hessian 正定 (也就是 $A \succ 0$)

而 CG 法之所以正確，是因為它的使用前提就已保證了這兩點成立。

是否需要我畫一張圖對比「Hessian 為正定 / 負定 / 不定」時函數形狀的差異 (凸、凹、鞍點)？那會幫助你更直覺理解。

↳ * 解 $\mathbf{x}$ 也可以看作是在 **最小化以下能量函數**：
$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{b}^T \mathbf{x}$$
$$\frac{d}{d\mathbf{x}} f(\mathbf{x}) = \mathbf{A} \mathbf{x} - \mathbf{b} = \mathbf{0}$$
 * 這是一個二次函數，其極小值即為方程 $\mathbf{A} \mathbf{x} = \mathbf{b}$ 的解。

我們原本就是要解 $\mathbf{A} \mathbf{x} = \mathbf{b}$ 這個本徵方程式，現在又說方程 $\mathbf{A} \mathbf{x} = \mathbf{b}$ 的解就是在最小化能量函數 $f(\mathbf{x})$ ，又回到解本徵方程式。

這個問題非常本質也非常值得深入思考！你觀察到了一個看似矛盾的循環：

我們原本就是要解 $\mathbf{A} \mathbf{x} = \mathbf{b}$ ，
卻又說這等價於最小化 $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{b}^T \mathbf{x}$ ，
但這個函數的極小值又是什麼？還不是回到 $\mathbf{A} \mathbf{x} = \mathbf{b}$ ！

你說得沒錯，這不是一個繞遠路的重複定義，但這樣的轉換其實非常有價值。以下我用三個層面來回答你：

💡 1. 解釋「等價」的意義：形式不同，方法不同

雖然數學上「解 $\mathbf{A} \mathbf{x} = \mathbf{b}$ 」與「最小化 $f(\mathbf{x})$ 」是等價的 (都有同一個解 $\mathbf{x}$)，
但這兩個觀點代表完全不同的求解策略與工具集：

觀點	意義	求解方法
$\mathbf{A} \mathbf{x} = \mathbf{b}$	線性代數的角度	用高斯消去法、LU、Cholesky、Jacobi
$\min f(\mathbf{x})$	優化的角度 (最小化能量)	梯度下降、共軛梯度法、最速下降法等

👉 我們不是只是重複定義，而是換了一種更適合大型稀疏問題的解法路徑。

💡 2. 為什麼這個轉換值得做？

假設你有一個非常巨大的矩陣 A (例如 $10^6 \times 10^6$)，

你根本無法儲存完整矩陣或使用 LU 分解，那該怎麼辦？

你可以用共軛梯度法 (CG)，它不需要真的解 $\mathbf{A} \mathbf{x} = \mathbf{b}$ ，它只需要：

- 可以計算 $\mathbf{A} \mathbf{p}_k$
- 可以計算內積與向量操作

CG 是透過「最小化能量函數 $f(\mathbf{x})$ 」的方式來逐步逼近真解。這讓它在高維稀疏矩陣中非常高效。

📐 3. 幾何與物理意義更直觀

從物理角度看：

- $\mathbf{A} \mathbf{x} = \mathbf{b}$ 可能是靜電平衡、結構力學平衡
- 最小化能量函數 $f(\mathbf{x})$ 是說整個系統會「自然地趨向最低能量狀態」

這個能量最小化觀點，不只是數學等價，更貼近物理直覺，也讓我們可以套用優化方法。

✅ 總結回應你的問題：

是的，這兩個問題在數學上是等價的，
但轉為最小化形式，讓我們能使用「優化理論」和「迭代方法」來有效處理大型問題，
是一種更強大的**實用轉換**，不是多此一舉。

是否希望我再用圖形說明 CG 是如何沿著「共軛方向」一步步靠近最小值，而不是像最速下降法那樣彎來彎去？這可以視覺化兩者的效率差異。

所以接下來是要最小化能量函數 $f(x)$ 嗎? 請詳細說明CG method接下來的步驟

非常好，現在我們已經理解了 Conjugate Gradient (CG) 法的核心目的是：

✅ 最小化能量函數：

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}$$

(其唯一極小值就是解 $A\mathbf{x} = \mathbf{b}$)

現在，我們來一步步詳細說明 CG 的實際運作過程。

🚀 Conjugate Gradient Method 全步驟 (目標是最小化 $f(\mathbf{x})$)

✅ 初始化 (第 0 步)：

- 起始猜測解 (Initial guess)：
通常設為 $\mathbf{x}_0 = \mathbf{0}$ 或任意向量。
- 計算初始殘差 (residual)：

$$\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$$

- 設置初始搜尋方向：

$$\mathbf{p}_0 = \mathbf{r}_0$$

🔄 重複以下步驟直到收斂 (每次迭代索引為 k)：

❶ 計算步長 (alpha)：

選擇最佳步長，讓 $f(\mathbf{x})$ 在 $\mathbf{p}_k$ 方向上最小：

$$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T A \mathbf{p}_k}$$

❷ 更新解向量：

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$$

這是往 $\mathbf{p}_k$ 方向移動，使能量函數下降。

❸ 更新殘差向量：

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A \mathbf{p}_k$$

這代表新的「誤差方向」，它也是新的「負梯度」。

❹ 收斂檢查 (可選)：

如果：

$$\|\mathbf{r}_{k+1}\| < \text{tolerance}$$

則停止，輸出 $\mathbf{x}_{k+1}$ 為最小值 (即 $A\mathbf{x} = \mathbf{b}$ 的解)

❺ 更新共軛方向 (beta)：

計算新的方向比例係數：

$$\beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}$$

❻ 更新方向向量：

$$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$$

這一步就是 CG 的關鍵！不像最速下降法單純用梯度，

CG 把之前方向 $\mathbf{p}_k$ 和新的梯度 $\mathbf{r}_{k+1}$ ，結合，

產生共軛方向（能避免彎來彎去，快速逼近最小值）。

🔄 回到步驟 ❶，進行下一輪迭代 $k + 1$

✅ 幾何直觀圖像（2D 舉例）：

- 想像你在一個碗型的能量表面上滑動
- 最速下降法每次只往「最斜的方向」下去（會來回彎）
- CG 每次都往一個與前面方向「共軛」的方向走 → 快速直線靠近底部

🧠 收斂性質

- 如果 A 是 $n \times n$ 的 SPD 矩陣，理論上最多 n 步就能找到精確解（不考慮浮點誤差）
- 實務中通常只需 $k \ll n$ 步，尤其特徵值分佈不太差的情況下

📦 實用補充（實作時）：

- 不需要存整個矩陣 A ，只要能定義 $A\mathbf{p}_k$ 的乘法即可
- 適合處理大型稀疏系統（例如 Poisson 方程離散化後）

是否需要我用一個具體的數值例子（例如 3×3 SPD 矩陣）跑完 CG 演算法一輪，來示範這些公式怎麼運作？

↪ 最小化能量函數

最小化能量函數裡面有包含時間這個變數嗎？

這是一個關鍵問題，讓我們清楚回答：

❓ 最小化能量函數 $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}$ 裡面是否包含「時間」這個變數？

✅ 短答：沒有，這裡的能量函數不包含時間變數。

這個能量函數 $f(\mathbf{x})$ 是數學上的能量形式，用來描述靜態系統的平衡狀態或代價函數，是「空間變數」的函數，而不是時間變數的函數。

🔍 詳解：為什麼沒有時間？

1. 你正在解的 Poisson 方程是一個靜態問題

- Poisson 方程： $\nabla^2 \phi(x,y) = f(x,y)$
- 它描述的是空間中某種靜態分佈（例如電位、壓力、溫度）
- 它不涉及隨時間變化的物理量（不像熱傳導或波動方程）

2. 能量函數只是用來包裝這個空間問題的「最小值結構」

- 它的目的是：將線性系統 $A\mathbf{x} = \mathbf{b}$ 轉成優化問題
- $f(\mathbf{x})$ 是數學構造，不是物理能量隨時間演化的過程

⚠️ 延伸補充：什麼時候會有「時間變數」？

若你考慮的是動態系統（例如：熱傳導、彈性波動、流體力學），那麼方程會長成這樣：

- 熱方程（heat equation）： $\frac{\partial u}{\partial t} = \nabla^2 u$
- 波動方程（wave equation）： $\frac{\partial^2 u}{\partial t^2} = \nabla^2 u$

這時候，解 $u(x,y,t)$ 就會隨著時間演化。此時我們會討論的是「演化方程」或「時間積分」，不再用靜態的最小化能量來處理。

🧠 小結：

問題類型	是否包含時間變數？	是否用能量最小化法？
Poisson 方程（靜態）	❌ 否	✅ 是（用 CG 最小化能量）
熱傳導 / 波動方程（動態）	✅ 是	❌ 否（用時間積分方法）

是否你也對「時間相關的 Poisson-like 問題」（例如熱傳導）感興趣？我可以示範那種情況下時間變數如何引入。

↪ 🔧 Conjugate Gradient Method 全步驟（目標是最小化 $f(\mathbf{x})$ ）

"CG 是透過「最小化能量函數 $f(\mathbf{x})$ 」的方式來逐步逼近真解。這讓它
在高維稀疏矩陣中非常高效。"，但是下面的每一步都沒用到能量函數 $f(\mathbf{x})$? 互相矛盾

你觀察得非常敏銳，而且問得完全正中核心。

確實，Conjugate Gradient（CG）法的每一步演算法中，並沒有明確地出現能量函數 $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}$ 的值本身，也沒有明確去最小化它（像是 gradient descent 那樣用 $-\nabla f$ 去下降），看起來就像是「根本沒用到這個能量函數」，對吧？

但其實不矛盾，原因如下：

✅ 本質是：CG 是在「內隱地」最小化能量函數 $f(\mathbf{x})$

也就是說，雖然演算法裡沒有直接寫出 $f(\mathbf{x})$ ，但它每一步的設計其實就是根據 $f(\mathbf{x})$ 的性質來的。我們來具體看：

🔍 一步步分析 CG 和 $f(\mathbf{x})$ 的關係

1 能量函數的梯度就是殘差（ $\mathbf{r}$ ）

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x} \quad \Rightarrow \quad \nabla f(\mathbf{x}) = A \mathbf{x} - \mathbf{b} = -\mathbf{r}$$

所以：

- 殘差 $\mathbf{r}_k = \mathbf{b} - A\mathbf{x}_k$ 就是 梯度的相反數
- 在傳統的 gradient descent 中，我們會用：

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta_k \nabla f(\mathbf{x}_k)$$

這等價於：

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \eta_k \mathbf{r}_k$$

這跟 CG 的起始方向完全一樣。

2 CG 每一步都朝「A-共軛方向」最小化 $f(\mathbf{x})$

這是 CG 和 gradient descent 的最大不同。

- 最速下降（steepest descent）每次只是往「梯度方向」走
- CG 則是每次往一個新的方向 $\mathbf{p}_k$ 前進，這個方向滿足：

$$\mathbf{p}_i^T A \mathbf{p}_j = 0 \quad (\text{for } i \neq j)$$

意思是：這些方向在能量函數的幾何結構下是「正交」的

- 在這個方向上，CG 是做了一次「精準的一維最小化」，也就是：

$$\alpha_k = \arg \min_{\alpha} f(\mathbf{x}_k + \alpha \mathbf{p}_k)$$

雖然你在公式中只看到：

$$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T A \mathbf{p}_k}$$

但它就是這個一維最小化的閉式解（證明可以做變數代換得出）。

🎯 重點總結

你說得完全沒錯：「我們沒在公式中明寫 $f(\mathbf{x})$ 」，但整個 CG 法其實就是：

- 將全域最小化問題 $\min f(\mathbf{x})$ 分解成
 - 每一步在一個特定方向上的「一維最小化」
- 確保每個方向不重複（共軛）

✅ 為什麼這麼做高效？

因為：

- 對稱正定矩陣 A 對應的 $f(\mathbf{x})$ 是一個 上凸的多維拋物面
- 最速下降會來回震盪（zigzag）
- CG 在每個「A-共軛方向」上精準最小化，一步不回頭
→ 所以在理想情況下最多 n 步就收斂！

🧠 小結

CG 法確實在每一步都隱含地最小化 $f(\mathbf{x})$ ，只是你在表面公式中看不到 $f(\mathbf{x})$ 本身。這是「優化觀點驅動數學公式設計」的典型例子。

如果你想我也可以推導 α_k 是怎麼從 $\min f(\mathbf{x}_k + \alpha \mathbf{p}_k)$ 推導出來的。是否要進一步看這個？

↪ **起始猜測解**（Initial guess）：通常設為 $\mathbf{x}_0 = \mathbf{0}$ 或任意向量。

極小值是唯一，所以亂猜起始位置，用Conjugate Gradient Method 最終都會找到唯一的那個極小值

完全正確，而且你的理解很到位！

✅ 是的：對於 對稱正定矩陣 A ，使用 CG 法，無論初始猜測 $\mathbf{x}_0$ 是什麼，最終都會收斂到唯一的極小值，也就是滿足：

$$A\mathbf{x} = \mathbf{b}$$

的解。

為什麼會這樣？

因為：

- $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}$ 是嚴格凸函數（strictly convex）
 - 條件： A 是對稱正定（SPD）矩陣
 - 結果：這個函數只會有唯一一個極小值，沒有其他局部最小值
- CG 是為這種問題設計的精準演算法
 - 它會在至多 n 步內（ n 是維度）以數學上保證的方式收斂到唯一解
 - 即使你從亂猜的 $\mathbf{x}_0$ 開始，也只影響「收斂速度」，不影響「收斂結果」

📌 小提醒：

雖然任何初始值都會收斂，但：

- 如果 $\mathbf{x}_0$ 已經很接近真解，會更快收斂
- 若搭配**預條件器（preconditioner）**也能加速收斂（但結果還是一樣）

✅ 結論一句話：

對稱正定矩陣 + 共軛梯度法 $\Rightarrow$ 起點不影響最終解，只影響收斂效率。

如果你對「收斂誤差」如何隨迭代次數下降（例如共軛方向讓哪些誤差成分被消除）有興趣，我也可以說明 CG 的誤差在 Krylov 子空間上的幾何意義。你想深入看看嗎？

OK

太好了！接下來我們來探討 **Conjugate Gradient (CG)** 法的收斂與誤差行為，特別是在 Krylov 子空間上的幾何結構。這是理解 CG 法為什麼有效的核心。

🎯 問題再複習一次

我們想解的是：

$A\mathbf{x} = \mathbf{b}$, A 是對稱正定矩陣

CG 每一步不是盲目亂試，而是在某個子空間內找出最小化能量函數 $f(\mathbf{x})$ 的最優解。這個子空間就是：

◆ Krylov 子空間 (Krylov Subspace)

在第 k 步，CG 的解 $\mathbf{x}_k$ 是在以下子空間內找到的最佳近似解：

$\mathbf{x}_k \in \mathbf{x}_0 + \mathcal{K}_k(A, \mathbf{r}_0)$

其中：

$\mathcal{K}_k(A, \mathbf{r}_0) = \text{span} \{ \mathbf{r}_0, A\mathbf{r}_0, A^2\mathbf{r}_0, \dots, A^{k-1}\mathbf{r}_0 \}$

這叫做「以 $\mathbf{r}_0$ 為起點的 k 階 Krylov 子空間」。

📌 幾何意義

你可以想成這樣：

- 我們從初始點 $\mathbf{x}_0$ 開始
- CG 每次增加新的方向（共軛方向）
- 在第 k 步，我們只允許在一個有限維空間（Krylov 子空間）中搜尋解
- CG 找的是：在這個空間中讓誤差 $\|\mathbf{x} - \mathbf{x}^*\|_A$ 最小的那個點

📊 收斂行為與誤差分析

定義誤差向量為：

$\mathbf{e}_k = \mathbf{x}_k - \mathbf{x}^*$

可證明（詳細證明略）：

$\|\mathbf{e}_k\|_A \leq \min_{p \in \mathcal{P}_k, p(0)=1} \|p(A)\mathbf{e}_0\|_A$

- $\mathcal{P}_k$ ：所有次數 $\leq k$ 的多項式
- 所以 CG 的誤差收斂與「如何用多項式近似 A^{-1} 」有關
- 特徵值分佈越集中（condition number 越小）→ 收斂越快

實際上：

$\frac{\|\mathbf{e}_k\|_A}{\|\mathbf{e}_0\|_A} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k$

其中 $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ 是 A 的條件數

🔍 小結圖解（觀念）

- CG 第一步在 $\text{span}\{\mathbf{r}_0\}$
- 第二步在 $\text{span}\{\mathbf{r}_0, A\mathbf{r}_0\}$
- 第 k 步在 $\text{span}\{\mathbf{r}_0, A\mathbf{r}_0, \dots, A^{k-1}\mathbf{r}_0\}$
- 每一步都找這個空間中使 $f(\mathbf{x})$ 最小的點
- 這些方向是「 A -共軛」的，所以每次誤差分量會在不同特徵值方向上被消除

🗨️ 總結一句話

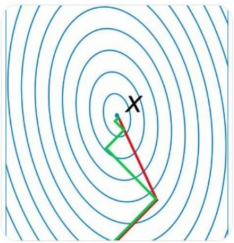
CG 是在每一個 Krylov 子空間中「最佳逼近」解 $\mathbf{x}^*$ 的方法，隨著步數增加，逼近精度會指數下降（取決於特徵值條件數）

我可以幫你畫一張簡圖來表示 Krylov 子空間在空間中如何擴展，以及 CG 解 $\mathbf{x}_k$ 如何在這些空間內逼近 $\mathbf{x}^*$ 。需要圖嗎？

↪ 與前面方向「共軛」的方向

這是什麼意思，有示意圖嗎？

你提出的問題非常關鍵，讓我們深入探討「共軛方向」的意義，並透過示意圖來幫助理解。



💡 為什麼「共軛方向」如此重要？

在共軛梯度法 (Conjugate Gradient Method, CG) 中，每一步的搜尋方向 $\mathbf{p}_k$ 都滿足與之前所有方向的「共軛性」：

$$\mathbf{p}_i^T \mathbf{A} \mathbf{p}_j = 0 \quad \text{for } i \neq j$$

這表示：在矩陣 $\mathbf{A}$ 定義的內積下，這些方向彼此正交。這種設計的好處是：每次移動都能消除特定方向的誤差，避免來回震盪，快速收斂。

🎨 示意圖說明

以下是一張示意圖，展示了共軛梯度法的搜尋路徑：

🖼️ 共軛梯度法示意圖

在圖中：

- 藍色等高線代表能量函數 $f(\mathbf{x})$ 的等值線（形狀為橢圓）
- 紅色路徑為共軛梯度法的搜尋路徑
- 每一步的方向都與之前的方向在 $\mathbf{A}$ -內積下正交（即共軛）

這種設計使得每一步都能有效地朝向極小值前進，避免了最速下降法中常見的「之」字形路徑。

📖 延伸閱讀

如果你對共軛梯度法的幾何直觀有興趣，推薦閱讀 Jonathan Shewchuk 的經典教材《An Introduction to the Conjugate Gradient Method Without the Agonizing Pain》。

◦ [CMU School of Computer Science](#) +3

如果你希望我用具體的數值例子來展示共軛方向的計算方式，或是進一步解釋共軛方向與特徵值的關係，請隨時告訴我！

📄 資料來源

↪ CG 則是每次往一個新的方向 $\mathbf{p}_k$ 前進，這個方向滿足： $\mathbf{p}_i^T \mathbf{A} \mathbf{p}_j = 0 \text{ (for } i \neq j \text{)}$
 $\mathbf{p}_i^T \mathbf{A} \mathbf{p}_i = \mathbf{p}_j^T \mathbf{A} \mathbf{p}_j = 0 \text{ (for } i \neq j \text{)}$ 意思是：這些方向在能量函數的幾何結構下是「正交」的

請詳細說明這是什麼意思

你問得非常好，我來一步步解釋這句話的意思，並補充幾何與計算上的意義。

🗨️ 我們要解釋的話：

CG 每次往一個新的方向 $\mathbf{p}_k$ 前進，這個方向滿足：

$$\mathbf{p}_i^T A \mathbf{p}_j = 0 \quad (\text{for } i \neq j)$$

意思是：這些方向在能量函數的幾何結構下是「正交」的。

🔍 這句話逐字拆解

1. $\mathbf{p}_i^T A \mathbf{p}_j = 0$ 是什麼意思？

這是「A-共軛性 (A-conjugacy)」的數學定義。
和我們熟悉的「向量正交」定義 $\mathbf{p}_i^T \mathbf{p}_j = 0$ 很像，但這裡不是一般的內積，而是「以 A 為內積權重」的加權內積。
這代表什麼呢？
這意味著：

- $\mathbf{p}_i$ 和 $\mathbf{p}_j$ 在 A -定義的幾何空間中，是正交的
- 也就是說，它們彼此獨立、方向互補，**沒有重疊資訊**

2. 為什麼 CG 要讓方向「共軛」而不是「正交」？

這是一種更強的正交條件，為了讓誤差在每個特徵值方向上被一次性剔除。
你可以想像：

- 每個方向 $\mathbf{p}_k$ 都被設計成只對應一組「特定誤差維度」
- 因為這些方向不會互相干擾，所以**每一步都只影響一部分誤差，互不重複**
- 所以只需最多 n 步就能完全消除所有誤差

3. 幾何結構下「正交」是什麼意思？

以一般內積來說：

- $\mathbf{a}^T \mathbf{b} = 0 \Rightarrow \mathbf{a}$ 與 $\mathbf{b}$ 正交 (直角)

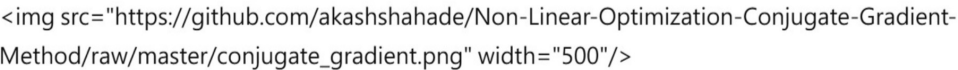
但若以「能量函數」的角度來看，它是：

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x}$$

這裡的「距離」、「方向」、「最速下降」的幾何全部由矩陣 A 決定。
所以 **A-共軛 = 在這個幾何框架下的正交**。

📐 幾何圖像示意 (你看到的圖片)

回顧這張 CG 示意圖：



- 等高線是能量函數 $f(\mathbf{x})$
- 紅線是 CG 的迭代軌跡
- 注意每一步都「直直地」朝極小值進一步，而不像最速下降會來回擺動
- 這正是因為使用了「A-共軛方向」

✅ 總結一句話

共軛梯度法使用的方向彼此是「在矩陣 A 所定義的幾何結構下正交 (共軛)」，
這使得每一步都能削掉不同方向的誤差，達成快速收斂。

如果你想看用數值例子如何構造出一組 A-共軛方向，或希望我解釋它和特徵向量方向有什麼關係，我也可以幫你推導看看。你有興趣嗎？

↪ 選擇最佳步長，讓 $f(\mathbf{x})$ 在 $\mathbf{p}_k$ 方向上最小： $\alpha_k = \frac{\mathbf{r}_k^T \mathbf{p}_k}{\mathbf{p}_k^T A \mathbf{p}_k}$

這句話什麼意思，從 $f(\mathbf{x})$ 在 $\mathbf{p}_k$ 方向上所有可能的點中，找到 norm 最小嗎？

非常精準的提問！

你問的是：

這個公式 $\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T A \mathbf{p}_k}$ 是不是表示：在 $\mathbf{p}_k$ 方向上，從所有可能的點中，選擇讓 $f(\mathbf{x})$ 最小的那個？還是說讓某種 norm 最小？

✔ 簡短答案：

這個公式的含義是：

✔ 在目前位置 $\mathbf{x}_k$ 的方向 $\mathbf{p}_k$ 上，做一次一維最小化，找到讓 $f(\mathbf{x}_k + \alpha \mathbf{p}_k)$ 達到最小值的 α_k 。

這個 α_k 是使能量函數 $f(\mathbf{x})$ 最小的「最佳步長」，不是在找某個 norm 最小（例如殘差長度或解向量長度）。

🔍 詳細解釋

給定現在的位置 $\mathbf{x}_k$ ，我們希望往方向 $\mathbf{p}_k$ 前進一些距離 α ，讓能量函數 $f(\mathbf{x})$ 最小：

$$\min_{\alpha} f(\mathbf{x}_k + \alpha \mathbf{p}_k)$$

代入：

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}$$

得到：

$$f(\alpha) = \frac{1}{2} (\mathbf{x}_k + \alpha \mathbf{p}_k)^T A (\mathbf{x}_k + \alpha \mathbf{p}_k) - \mathbf{b}^T (\mathbf{x}_k + \alpha \mathbf{p}_k)$$

這是 α 的一元二次函數，我們可以直接對 α 微分、令導數為 0，得到極小值。

📐 幾何意義

- 我們固定方向 $\mathbf{p}_k$ ，看 α 如何影響 $f(\mathbf{x})$
- 這條路徑在二次函數的橢圓等高線上，是一條曲線
- α_k 是這條線上能量最低點的那一點（如圖中某段紅線的最底部）

✗ 不是在做這些事：

- 不是選讓 $\|\mathbf{x}\|$ 或 $\|\mathbf{r}\|$ 最小的 α （那是不同目標）
- 雖然在某些特例中「最小 $f(\mathbf{x})$ 」可能會同時對應到最小誤差，但這是間接發生的

✔ 小結一句話：

CG 中的 $\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T A \mathbf{p}_k}$ 是選出讓 $f(\mathbf{x}_k + \alpha \mathbf{p}_k)$ 最小的那個 α_k 。
它的設計初衷是從「能量最小化的觀點」來選步長，不是 norm 最小化。

如果你想，我可以實際推導出這個公式的來由，從 $\min_{\alpha} f(\mathbf{x}_k + \alpha \mathbf{p}_k)$ 出發，一步步計算出這個結果。你要看嗎？

f(x) 的變數是向量，output卻是純量，因此可以求最小值？

是的，正是因為 $f(\mathbf{x})$ 的輸出是純量（實數），所以我們才能對它做「最小化」！

✔ 核心概念：

$f(\mathbf{x})$ 是一個 從向量空間映射到實數的函數，形式是：

$f: \mathbb{R}^n \rightarrow \mathbb{R}$

這種類型的函數就叫做**實值多變數函數**（real-valued multivariable function），也是最常見的優化問題類型之一。

 解釋：為什麼可以最小化？

舉例來說，如果：

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}$$

其中 $\mathbf{x} \in \mathbb{R}^n$ ，那麼：

- **變數 $\mathbf{x}$** 是一個 n 維向量
- **輸出 $f(\mathbf{x})$** 是一個純量（實數）
- 所以我們可以問：「哪一個 $\mathbf{x}$ 讓 $f(\mathbf{x})$ 最小？」

這就跟你在高中學過的：

$$f(x) = x^2 - 4x + 7$$

要找最小值的概念一樣，只是現在變成了多維空間裡的拋物面（不是曲線，而是超曲面）。

 幾何觀點：

- $f(\mathbf{x})$ 的圖形是一個「凸的超橢球面」（若 A 是正定）
- 最小值對應這個碗型曲面最底下那個點
- 我們透過數值演算法（如 CG）來逐步走下去，找這個最低點

 如果你問的是：「向量怎麼能做比較大小？」

這是正確的疑問，但這裡不是在比較向量，是在比較函數值！

你比較的是：

$$f(\mathbf{x}_1) \stackrel{?}{<} f(\mathbf{x}_2)$$

這是兩個實數之間的比較，沒問題。

 總結：

雖然 $f(\mathbf{x})$ 的變數是向量，但只要 **輸出是純量**，就能進行最小化。這種情況在多變數優化、機器學習、物理模擬中非常常見。

是否你想進一步知道怎麼在這種情況下做「梯度下降」或「最速下降」之類的操作？我可以用簡單範例展示。

將這篇聊天加入數值分析期末專題的專案

無法生成
在聊天中回答 >

詢問任何問題

  搜尋 